4-bit Array Multiplier: RTL to GDSII on SKY130

Simon Shibu (2026203003) | Akshat Agarwal (2026203014)
MTech VLSI, IIIT Hyderabad | Digital VLSI Assignment, October 2026
Repository: `mult4_2026203003`

1 Introduction

This report documents the design, verification and implementation of a 4-bit $\times$ 4-bit unsigned array multiplier (8-bit product, registered inputs and output, latency 2 clock cycles) on the SkyWater SKY130 process with the `sky130_fd_sc_hd` standard-cell library. The design was taken through RTL simulation (Part A), logic synthesis (Part B), static timing analysis (Part C) and place and route to a DRC- and LVS-clean GDSII layout (Part D). The results are used to answer the six questions in Section 6.

All tools ran in the IIC-OSIC-TOOLS Docker image on macOS (Apple Silicon): Icarus Verilog 14.0, GTKWave, Yosys 0.69, OpenSTA 3.1.0, LibreLane 3.1.0.dev3 (OpenROAD, Magic, Netgen) and KLayout 0.30.12, with the `sky130A` PDK. Every command is listed in order in the `README.md` of the repository.

Use of AI tools. I used AI tools during this assignment for guidance on the Docker and EDA setup, failure diagnosis and for drafting this report.

2 Design and verification (Part A)

Design. `full_adder` is written with `assign`: `sum = a ^ b ^ cin` and `cout = (a & b) | (cin & (a ^ b))`. `mult_array` registers `a` and `b` into 4-bit registers `a_r`, `b_r` (asynchronous active-low reset), forms the partial products $pp_i[j] = a_r[j] \wedge b_r[i]$ with 16 AND gates, and adds them row by row with 12 `full_adder` instances (rows 1–3, four adders each, carry-in of the first adder of each row tied to 0). Row 0 passes $r_0 = \{0, pp_0[3:1]\}$ down and gives $p[0] = pp_0[0]$; rows 1–3 give $p[1], p[2], p[3]$ as their lowest sum bit, and row 3 gives $p[7:4] = \{c_3[4], s_3[3:1]\}$. The 8-bit result is registered into `p`, so there are $4+4+8 = 16$ flip-flops and the latency is 2 clock cycles. The $\star$ operator is not used in the RTL.

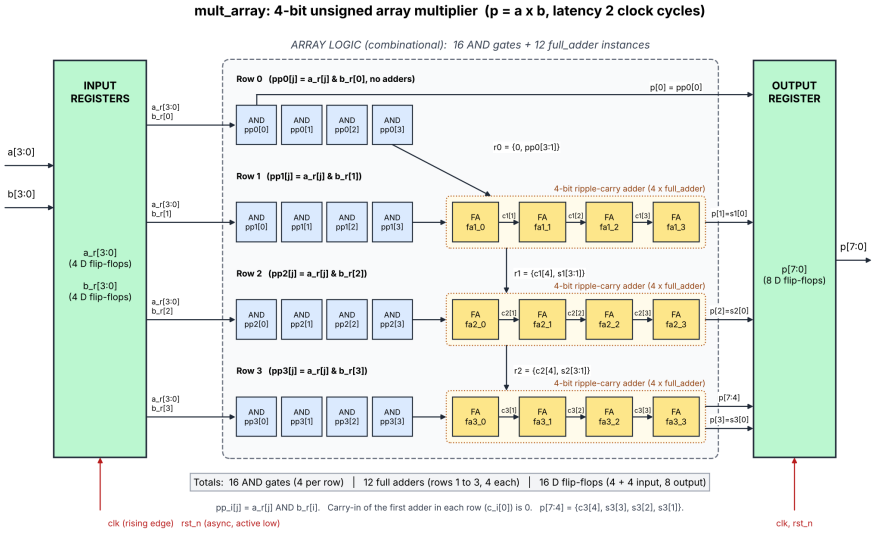


Figure 1: Block diagram of the multiplier: input registers `a_r`, `b_r` (8 flip-flops), 16 AND gates, 12 full adders in three rows and the 8-bit output register `p` (8 flip-flops).

Verification. `tb_mult` generates a 10 ns clock, holds `rst_n` low for the first 2 cycles, applies all 256 combinations of `a` and `b`, waits two rising edges (plus 3 ns for the gate-level cell delays) and compares `p` with `a * b`. It prints one line per mismatch and one summary line, and writes `mult.vcd`.

```
PASS: 256 vectors checked, 0 errors
```

Error test. With `sum` in `full_adder.v` temporarily changed to `a ^ b`, the testbench prints (then the change was undone and PASS was confirmed again):

```
FAIL: 256 vectors checked, 95 errors
```

Hand-worked example. My example uses $A = 3+5 = 8$ (last digit of the roll number plus 5) and $B = 7$. The hand calculation is in Figure 2: $8 \times 7 = 56 = 0011\,1000_2$. The same multiplication is found in the waveform in Figure 3: after the inputs $a = 8$, $b = 7$ are applied, they are captured at clock edge 1 and the product $p = 56$ appears after clock edge 2, which shows the 2-cycle latency.

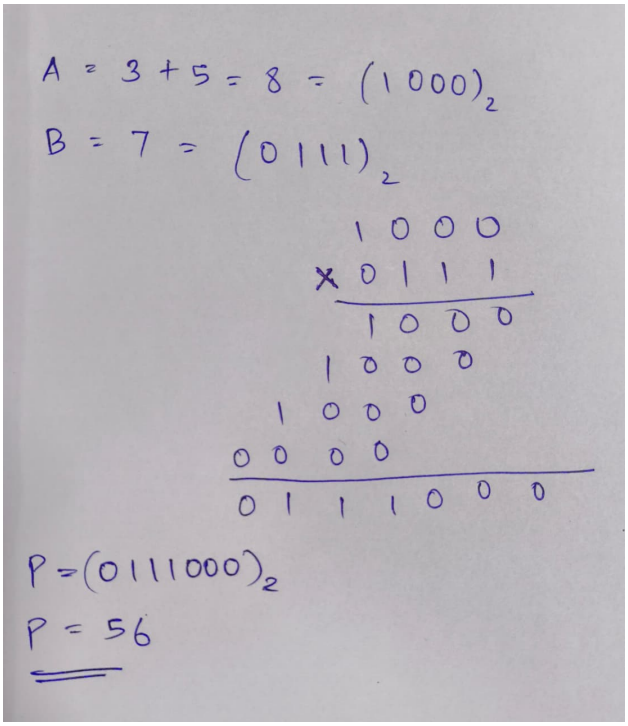


Figure 2: Hand-worked multiplication $A = 8$ (1000_2), $B = 7$ (0111_2): partial products, row-by-row sums and carries, and the product $56 = 0011\,1000_2$.

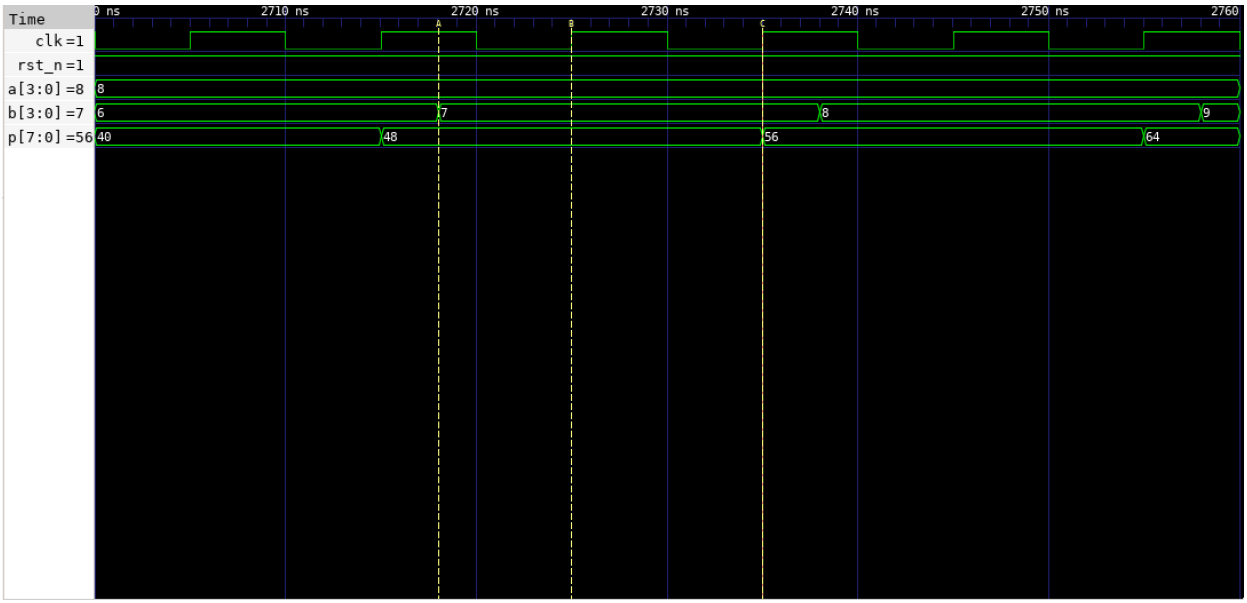


Figure 3: GTKWave waveform of `clk`, `rst_n`, `a`, `b` and `p` for 8×7. The inputs are captured at clock edge 1 and the product 56 appears at clock edge 2 (latency = 2 clock cycles = 20 ns).

3 Synthesis (Part B)

`synth.tcl` runs `read_verilog`, `synth -top mult_array -flatten`, `dfflibmap`, `abc`, `opt_clean -purge`, `stat -liberty` and `write_verilog`. Table 1 gives the results of `mult_array_stat.rpt` (the full report is in Appendix B). The synthesised netlist was simulated with the same testbench and prints `PASS: 256 vectors checked, 0 errors`, so synthesis did not change the behaviour. The first gate-level run failed because the testbench compared `p` 1 ns after the clock edge while the cell models with `UNIT_DELAY=##1` update their outputs 1 ns after the edge; the compare delay was changed to 3 ns.

Table 1: Part B results (Yosys, `sky130_fd_sc_hd`).

Metric	Value
Total number of cells	67
Number of flip-flops (`dfrtp_1`)	16
Total cell area	746.966 µm^2
Flip-flop area	400.384 µm^2
Flip-flop area (% of total)	53.6 %
Three most-used cell types	`dfrtp_1`: 16, `nand2_1`: 12, `xnor2_1`: 7
Netlist simulation result	PASS: 256 vectors checked, 0 errors

4 Timing (Part C)

`sta.tcl` reads the typical-corner library (`tt_025C_1v80`) and the netlist, defines the clock with period T, sets 1.0 ns input and output delays, and reports the worst setup slack and the critical path (`report_checks -path_delay max`). At $T = 10$ ns the worst slack is 7.396 ns, so $T_{\min} = 10 - 7.396 = 2.604$ ns and $F_{\max} = 1000/2.604 = 384.0$ MHz. Running again at $T = 2.6$ ns (rounded to 0.1 ns) gives a slack of -0.004 ns, i.e. close to 0 as expected. The latency is $2 \times T_{\min} = 5.208$ ns and the throughput equals $F_{\max}$.

Table 2: Part C results (OpenSTA, typical corner, ideal clock and wires).

Metric	Value
Worst slack at $T = 10$ ns	7.396 ns
$T_{\min}$	2.604 ns
$F_{\max}$	384.0 MHz
Slack at $T = 2.6$ ns	-0.004 ns
Critical path start/end	`_106_` (`b_r[0]`) / `_100_` (`p[6]`
Number of cells on the critical path	8 (plus the two flip-flops)
Latency ($2 \times T_{\min}$)	5.208 ns
Throughput	384.0 million results/s

Table 3: Part C - Critical path from flip-flop _106_ to flip-flop _100_.

#	Cell instance	Cell type	Delay (ns)	Arrival (ns)
0	_106_ (CLK→Q)	dfrtp_1	0.388	0.388
1	_043_	and2_0	0.180	0.568
2	_056_	or3_1	0.391	0.959
3	_058_	a32oi_1	0.250	1.208
4	_072_	o21ba_1	0.198	1.406
5	_073_	xnor2_1	0.147	1.553
6	_082_	maj3_1	0.385	1.937
7	_084_	maj3_1	0.358	2.295
8	_088_	xnor3_1	0.175	2.471

5 Physical design (Part D)

Configuration. CLOCK_PERIOD was chosen as $1.2 \times T_{\min} = 3.1248$ ns, rounded up to the next 0.5 ns, i.e. 3.5 ns, with FP_SIZING = "absolute" and a fixed 150×150 µm die. Timing failed in the slow corner (ss_100C_1v60) for every period below 7.0 ns, so the period was raised by 0.5 ns after each run. Eight runs were needed. The slack is not monotonic with the period (5.5 ns is better than 6.0 ns), because the tools optimise on a faster corner and the slow corner is only checked at signoff.

Table 4: All LibreLane attempts: clock period and worst setup slack (timing__setup__ws).

Run	1	2	3	4	5	6	7	8
Period (ns)	3.5	4.0	4.5	5.0	5.5	6.0	6.5	7.0
Slack (ns)	−2.118	−1.594	−1.123	−0.622	−0.122	−0.166	−0.035	+0.465
Result	fail	fail	fail	fail	fail	fail	fail	pass

Results. The final run (7.0 ns) finished with 0 DRC errors (Magic and KLayout), 0 LVS errors and a worst setup slack of +0.465 ns (corner max_ss_100C_1v60). The post-route maximum frequency is $F_{\max} = 1000/(7.0 - 0.465) = 153.0$ MHz (316.4 MHz at the typical corner, 384.0 MHz in Part C). The flow's own synthesis step produced 61 cells (734.454 µm^2), the final layout has 1883 cells. Table 5 summarises the results and the cell types that were added.

Table 5: Part D results (LibreLane, final run at CLOCK_PERIOD = 7.0 ns).

Metric	Value
CLOCK_PERIOD used	7.0 ns
Number of runs needed	8
Core area	17759.5 µm^2 (die 150×150 µm)
Cells after synthesis (Part B)	67
Cells after place and route	1883
Kinds of cells added	tap 245, decap 1299, fill 256, clock buffers clkbuf_16 3, port buffers buf_2 17, fanout buffers clkdlybuf4s50_1 2
Wirelength	2333 µm
Setup slack	+0.465 ns
Post-route $F_{\max}$	153.0 MHz
$F_{\max}$ from Part C	384.0 MHz
DRC errors	0
LVS errors	0

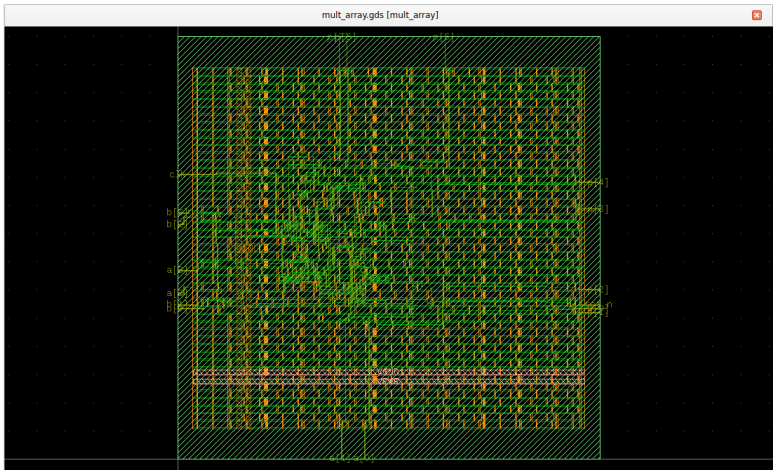


Figure 4: Final layout (`mult_array.gds`) in KLayout: the whole 150×150 µm die.

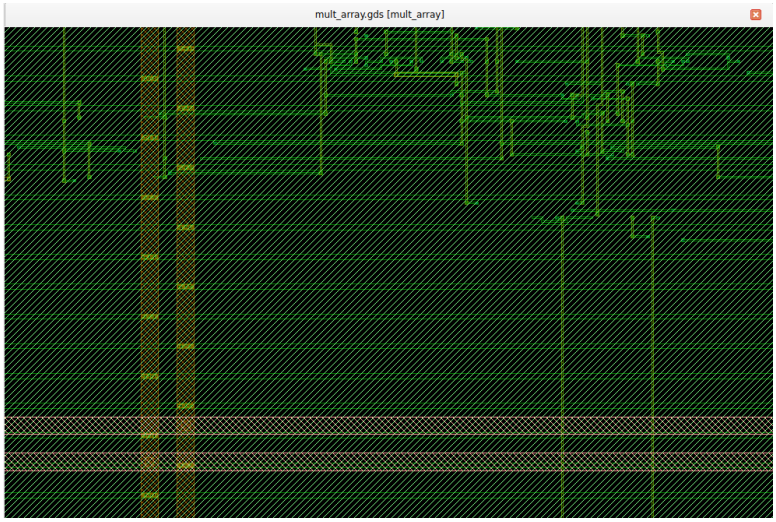


Figure 5: Zoomed view of the final layout

6 Answers to the questions

Q1. Logic cells. Synthesis produced 67 cells, of which 16 are flip-flops, so the logic is $67 - 16 = 51$ cells, not $16 + 12 = 28$. Yosys flattens the full adders and ABC maps the whole network to SKY130 gates, so a full adder is not one cell: it becomes several (for example `xnor2_1`, `xnor3_1` and `maj3_1`), and the ANDs are mostly merged into the following logic as NAND, `a21oi` and `o21ai` cells (12 `nand2_1` are used). Most SKY130 gates are inverting, so extra inverting cells appear too. The cell count therefore depends on how ABC optimises, not on the number of blocks in the RTL.

Q2. Critical path. The critical path runs from the input register `b_r[0]` (flip-flop `_106_`) to the output register bit `p[6]` (flip-flop `_100_`). It has 8 combinational cells (Table 3) and its delay at $T = 10$ ns is 2.471 ns: 0.388 ns clock-to-Q plus 2.083 ns of logic, and the capturing flip-flop needs 0.134 ns setup, so $T_{\min} = 0.388 + 2.083 + 0.134 = 2.605$ ns, matching 2.604 ns from the slack. The estimate of about $2N = 8$ full-adder delays agrees with the 8 cells on the path, but the cells are not all adder cells (four are partial-product and first-row logic), and the average delay per cell is $2.083/8 = 0.26$ ns, shorter than the 0.36–0.39 ns of one `maj3_1` on this path.

Q3. Flip-flop area. The 16 flip-flops take 400.384 µm^2 of the 746.966 µm^2 total, which is 53.6 %. The percentage is large because the design is small: the logic is only 51 cells (346.582 µm^2) while the 16 registers (4 + 4 input bits and 8 output bits) are needed regardless of the logic, and a resettable flip-flop is a big cell: one `dfrtp_1` is 25.024 µm^2, about 6.7 times a `nand2_1` (3.754 µm^2).

Q4. Post-route $F_{\max}$ and added cells. The post-route $F_{\max}$ is 153.0 MHz at the limiting corner, lower than

384.0 MHz from Part C. Part C uses the typical corner only, with ideal wires and an ideal clock. After place and route the timing includes the resistance and capacitance of the real wires (2333 µm of wirelength), clock skew (up to about 0.27 ns) and the port buffers, and the signoff checks the slow corner, where the cells are much slower. The flow went from 61 cells after its own synthesis to 1883 cells, i.e. added 1822 cells: *245 tap cells* (`tapvpwrvgnd_1`) connect the n-wells and the substrate to the supply at regular distances to prevent latch-up; *1299 decap cells* give local charge and reduce supply droop; *256 fill cells* fill the empty places in the rows so that the rails and wells are continuous and the density rules are met; *3 clock buffers* (`clkbuf_16`) from clock tree synthesis distribute the clock to the 16 flip-flops with low skew; *17 `buf_2` buffers* (9 input and 8 output buffers) isolate the ports and drive the pin and wire loads; and *2 `clkdlybuf4s50_1` buffers* repair the fanout of the reset net, which drives all 16 flip-flops (the log reports no hold violations).

Q5. DRC and LVS. A DRC (design rule check) checks the layout geometry against the foundry's rules: minimum width and spacing of the shapes in each layer, via enclosure, density, antenna rules and so on, so that the layout can be manufactured with a good yield. An LVS (layout versus schematic) check extracts the transistors and connections from the layout and compares them with the gate-level netlist, so it verifies that the drawn chip is the intended circuit (same devices and connections, no shorts or opens). Both must pass: a DRC-clean layout of the wrong circuit is useless, and a correct but rule-violating layout may not be manufacturable. The final run has 0 DRC errors (Magic and KLayout) and 0 LVS errors.

Q6. An 8-bit multiplier. With the formulas of the guide, an N-bit array multiplier has N^2 AND gates and $N(N-1)$ full adders, and its critical path is about $2N$ full-adder delays. For $N=4$ this gives 16 ANDs and $4\times 3=12$ full adders; for $N=8$ it gives 64 ANDs and $8\times 7=56$ full adders, which is $56/12=4.7$ times more (area grows about as N^2). The delay grows from $2\times 4=8$ to $2\times 8=16$ full-adder delays, i.e. it doubles (linear in N). With my numbers, the logic delay of 2.083 ns for 8 full-adder delays is 0.26 ns each, so the 8-bit logic delay is about $16\times 0.26=4.17$ ns; adding clock-to-Q (0.388 ns) and setup (0.134 ns) gives $T_{\min}\approx 4.69$ ns, i.e. $F_{\max}\approx 1000/4.69=213$ MHz instead of 384.0 MHz. The number of flip-flops would double from 16 to 32.

7 Conclusion

A 4-bit unsigned array multiplier (16 AND gates, 12 full adders, 16 flip-flops, latency 2 cycles) was verified exhaustively (256 vectors, PASS), synthesised to 67 cells and 746.966 µm^2, and found to run at up to 384.0 MHz ($T_{\min}=2.604$ ns) in the ideal typical-corner analysis. Taken through place and route with LibreLane at a 7.0 ns clock period (8 runs), it gives a DRC- and LVS-clean layout of 1883 cells with a post-route $F_{\max}$ of 153.0 MHz at the slowest corner. The drop from 384.0 MHz to 153.0 MHz comes from wires, clock skew, port buffers and the slow corner, and 1822 of the 1883 cells are physical or support cells (tap, decap, fill, buffers). The repository with all scripts, reports, the `README.md` and the GDSII file is `mult4_2026203003`.

Appendix A: Cell Count

```
    908 sky130_fd_sc_hd__decap_12
    245 sky130_fd_sc_hd__tapvpwrvgnd_1
    213 sky130_fd_sc_hd__fill_1
    179 sky130_fd_sc_hd__decap_6
    123 sky130_fd_sc_hd__decap_3
     56 sky130_fd_sc_hd__decap_8
     43 sky130_fd_sc_hd__fill_2
     33 sky130_fd_sc_hd__decap_4
     17 sky130_fd_sc_hd__buf_2
     16 sky130_fd_sc_hd__dfrtp_1
     13 sky130_fd_sc_hd__nand2_1
      5 sky130_fd_sc_hd__xnor2_1
      4 sky130_fd_sc_hd__maj3_1
      3 sky130_fd_sc_hd__xor2_1
      3 sky130_fd_sc_hd__xnor3_1
      3 sky130_fd_sc_hd__nand4_1
      3 sky130_fd_sc_hd__clkbuf_16
      2 sky130_fd_sc_hd__clkdlybuf4s50_1
      2 sky130_fd_sc_hd__and2_0
      2 sky130_fd_sc_hd__a21oi_1
      2 sky130_fd_sc_hd__a21o_1
      1 sky130_fd_sc_hd__or3_1
      1 sky130_fd_sc_hd__o21ai_0
      1 sky130_fd_sc_hd__nand3_1
      1 sky130_fd_sc_hd__mux2i_1
      1 sky130_fd_sc_hd__inv_1
      1 sky130_fd_sc_hd__a31oi_1
      1 sky130_fd_sc_hd__a22oi_1
      1 sky130_fd_sc_hd__a21boi_0
```

A Appendix B: critical-path report (OpenSTA, $T = 2.6$ ns)

```
Path Group: clk
Path Type: max

   Delay     Time   Description
---------------------------------------------------------
   0.000    0.000   clock clk (rise edge)
   0.000    0.000   clock network delay (ideal)
   0.000    0.000 ^ _106_/CLK (sky130_fd_sc_hd__dfrtp_1)
   0.388    0.388 v _106_/Q (sky130_fd_sc_hd__dfrtp_1)
   0.180    0.568 v _043_/X (sky130_fd_sc_hd__and2_0)
   0.391    0.959 v _056_/X (sky130_fd_sc_hd__or3_1)
   0.250    1.208 ^ _058_/Y (sky130_fd_sc_hd__a32oi_1)
   0.198    1.406 v _072_/X (sky130_fd_sc_hd__o21ba_1)
   0.147    1.553 v _073_/Y (sky130_fd_sc_hd__xnor2_1)
   0.385    1.937 v _082_/X (sky130_fd_sc_hd__maj3_1)
   0.358    2.295 v _084_/X (sky130_fd_sc_hd__maj3_1)
   0.175    2.471 v _088_/X (sky130_fd_sc_hd__xnor3_1)
   0.000    2.471 v _100_/D (sky130_fd_sc_hd__dfrtp_1)
            2.471   data arrival time

   2.600    2.600   clock clk (rise edge)
   0.000    2.600   clock network delay (ideal)
   0.000    2.600   clock reconvergence pessimism
            2.600 ^ _100_/CLK (sky130_fd_sc_hd__dfrtp_1)
  -0.134    2.466   library setup time
            2.466   data required time
---------------------------------------------------------
            2.466   data required time
           -2.471   data arrival time
---------------------------------------------------------
           -0.004   slack (VIOLATED)
```